

4. Try Shortcuts

How? Based on your experience, you can make educated guesses about which areas of your program's source relate to your bug's symptoms. You can also check systems that have repeatedly had bugs and confirm they're not at fault, initially ignoring the ones with responsibilities that probably don't relate to your particular problem.

Why? Once you know you need to jump to a specific point in your program's sources, getting there doesn't take much time: With most integrated development environments or editors, it takes only a moment to jump from line 20 to line 10,438 in a file. One method of identifying the part you need is narrowing down the target area by ruling out larger sections. If you can disregard most of the program's sources, you'll have a better idea of where to look for your target line. The presented shortcuts can work either way: discovering that a system is related to your bug or ruling it out.

Shortcuts are alluring. It's difficult to resist skipping all the hard work and getting a task done a lot quicker than your team would typically expect. However, you must be careful not to fall into the trap of repeatedly trying to find shortcuts without making any progress. Likewise, it's easy to get too focused on one where the line of inquiry doesn't lead to the spot that's at fault. If shortcuts don't quickly get you where you need to go, they slow you down because they keep you from beginning detailed investigations. In the worst case, you're mistaken and spend time looking into a supposed lead that turns out to be a dead end.

Throughout your work, keep checking to make sure you're on the right track. Does the spot you found via a shortcut really fit the bug you're working on? Is it reasonable to conclude that this code area is related to the observed symptoms? Or might this code be unrelated, meaning you can rule out this area as the source of the problem? Make sure to stay focused.

IN PRACTICE: AVOID UNLIKELY CULPRITS

On one project, my colleague was working on a bug related to network communication: Something was going wrong with our HTTP requests. The project used curl, a library that implements several network protocols. My colleague followed the bug's traces and spent a lot of time trying to debug curl itself because that seemed like the logical source of the problem.

However, libraries like curl have been developed over numerous years, creating highly optimized, complex code that can be hard to read and understand if you haven't dealt with it before. Ultimately, it's more likely that the bug is somewhere in the usage of the library rather than in the library itself. Indeed, my colleague's bug turned out to be hiding in our code.

Using shortcuts isn't always simple; it may even be impossible. It requires some experience with software development in general and with the current project. The more experience and knowledge you gather in both areas, the more valuable this strategy will be.

If you can't nail the spot responsible for your bug by using shortcuts, you can still learn something from where they take you. You can ignore unrelated areas during the remainder of your investigation and employ other strategies on vaguely related systems. For example, if you find a function somehow related to the bug, you can use a debugger to put a breakpoint in it and drill down further from there (see [“Debug the Code”](#) on [page XX](#)).

The following sections introduce several possible shortcut methods. They may very well fall short, but if they're successful, they'll let you jump directly to the right point without any deeper investigation.

Make Educated Guesses

You might start by making an educated guess about the problem's location. In a perfect situation, you would have encountered this exact bug before, but you can't rely on that every time. However, if you're familiar with the code base, this particular kind of bug, or debugging in general, you may be able to make a confident guess. The following sections outline several sources of experience.

A Particular Program

If you've worked on a specific program for a long time, you'll get to know its idiosyncrasies and how it reacts to certain kinds of bugs. You may be able to determine a bug's root cause and location from its symptoms alone. This is more likely with bugs similar to those you've worked on before, but it's also possible for bugs that interact with systems you know.

Understanding the program's overall architecture can provide information about how it works in general. Once you start investigating a bug and getting to know its surroundings, connected systems, and related functionality, you'll begin building a mental model of its context, which makes it easier for you to navigate the program's sources and get to any relevant point quicker.

Knowing fundamental decisions from the program's development and details about its technical debt also provides insight into approaching a bug. If you're familiar with the program's long-lasting temporary workarounds or hacky solutions to previous bugs, you have some great starting points to investigate because you have good reason to be suspicious of them.

IN PRACTICE: QUESTION YOUR ASSUMPTIONS

I once had to fix a seemingly simple bug in a tool my organization used internally. The program wasn't super complex, the bug report was very clear, and the requested change didn't seem outrageously complicated. When I started debugging the tool, however, none of the breakpoints I had previously set up seemed to do anything.

Then I started investigating my development setup. Surely something must have been wrong with the IDE, or maybe I was using the wrong build configuration without debugging information? I spent a lot of time checking everything around the program, but I couldn't find any problems.

Eventually, I took a step back and examined what the program did during its launch. I discovered that the initial launch executable didn't do much. It just checked whether an update was available and then launched a different executable, which I wasn't debugging. This piece allowed me to simply launch the other executable and start work. From then on, I was a lot quicker at fixing bugs in this tool.

Common Functionality

In addition to knowing a particular program, if you've worked on similar functionality before, you probably know how it can break. For example, you might know what can cut off sounds, why GUI elements may not adjust properly when resizing a window, or why a command line tool might misinterpret a filepath with spaces in it. When a bug occurs in a comparable functionality, you probably have at least an inkling of what's going on and where to start looking, even if you don't have a solution ready. This experience gives you an advantage across different projects.

IN PRACTICE: DERIVE LEADS FROM INCORRECT BEHAVIOR

Whenever I see a blank space, the outline of a box, or even a box with a question mark inside replacing a character in the middle of text, I have a good idea about what's going on. Because I've seen this problem several times already, whether I'm working on a web application, desktop GUI software, or a 3D program that uses a bespoke implementation to render text, I know two facts for sure. First, that character must be out of the ordinary—that is, not ASCII. Second, the font used for rendering must not support the character. In addition, the function decoding the text might not handle special characters correctly. All this provides me with multiple avenues of investigation, one of which will likely lead to the bug's root cause.

Lots of functionality is based on the same ideas and implementation principles. Ultimately, if a piece of code fulfills the same purpose as another, it likely works similarly. The details contain many differences, but similar functionality can lead to similar bugs.

Popular Frameworks

You can apply your experience with popular frameworks across multiple projects. Frameworks like Electron, React.js, and WinForms make developers' lives easier because they provide a basis to build programs. Knowing a program's framework tells you what kind of restrictions it imposes. It may also familiarize you with potential usage pitfalls that can lead to bugs, how bugs manifest with the framework present and what tools it offers to track them down, and the framework's own weaknesses and how to avoid them. If you can't avoid these problems, you may want to put them on your list of suspicious systems to investigate first whenever a bug is reported.

The significant differences in how frameworks are developed may influence how you approach a bug inside or connected to one. With the source code available to you, you can closely analyze it to determine why something breaks. If the framework developer doesn't share the source, it'll be impossible to fix a bug directly, but you can still use your knowledge to work around such an issue or ask the developer for a fix.

A company may develop bespoke frameworks to use internally and share across multiple projects. While knowing about those is of limited use at other organizations, you can use your experience when transferring internally. You may also be able to apply concepts you've learned from bespoke frameworks to others because their implementation may be similar if the functionality is comparable.

Foundational APIs

Programs and frameworks with similar purposes are written in similar ways partly because their systems must interact with the same foundational APIs. Knowing these APIs and their potential problems allows you to avoid, identify, and fix related bugs.

A program must interact with the same operating systems for input handling, window creation, multi-threading, and so on. It most likely uses the programming language's standard library for all kinds of purposes, such as containers, networking, and file access. If the software uses a database, it must interact with a well-established implementation such as InfluxDB, MongoDB, MySQL, or PostgreSQL. Programs that support rendering 3D scenes must use a graphics API like Direct3D or Vulkan.

While frameworks often abstract away the interaction with foundational APIs, their peculiarities and limitations can still leak through, exposing you to the same behavior and properties—and potentially the same bugs. If an API has particular challenges, developers will likely make the same mistakes or overlook the same unexpected activity when implementing the code that uses it. Complex pre- and post-conditions may also make an API really difficult.

IN PRACTICE: BEWARE OF AMBIGUOUS NAMES

A platform I once worked on offered an API to play audio samples. The function in question had the following signature:

```
void play_samples(void* data, size_t length, size_t sample_bits)
```

I expected the second parameter to specify the length of the data in bytes, which I happily provided. The result was an audio track that sounded good for a while but had a lot of noise toward the end.

It turned out that the API actually required me to provide the number of samples for the second parameter. While the number of samples relates to size in bytes, it's not the same in most cases. For a 16-bit sample, the size in bytes would be 2, but the sample count would be 1. That's why the audio initially sounded fine, then played noise for the same duration. Admittedly, the documentation probably specified that length must be the sample count, but looking at the function's signature made it easy to jump to the wrong conclusion.

The next time I work on this platform and encounter a bug related to audio playback, I'll check this particular function first. Learning about an easily mis-used function like this one makes it easier to fix similar bugs in the future.

Common Patterns

In addition to programs being influenced by their interaction with common APIs, many have similar functionality, which leads to shared

implementation patterns. (Some books even catalog and proliferate those designs.) Even if you don't intentionally use specific patterns, your architectural decisions are likely influenced by other software you've seen, the source code you've read, and any code you've written. Ultimately, you can match any designs you've seen before, especially those you've used, with your mental model to quickly understand a particular implementation. You'll also know some of the pattern's shortcomings and potential pitfalls that may cause bugs.

For example, the *model-view-controller (MVC) pattern* divides responsibilities based on the data, its display, and the logic applied to it. Most programs that have the terms *model*, *view*, and *controller* in the code use this design. Knowing that means you have a general idea of how the code is built and which bugs can potentially hide around its use, such as the way its relatively static dependency chain can make updating the view inefficient. That makes it a lot easier to find an entry point in the code for a bug where the view always lags.

IN PRACTICE: MIND INSTANTANEOUS INPUT EVENTS

I've often had to deal with missing user input, but that problem isn't based on a popular pattern. It comes from low-level input handling interfaces, which work in one of two ways. Either the program polls for the state of the relevant keys and sensors or it listens to events that the operating system generates. Bugs that lose input must be connected to misusing one of these paradigms and usually result from brief holding time or quick repetition.

If the code polls the input state periodically, it's likely to miss actions that don't last long enough or fail to capture state changes that occur between two polls. If the program listens to events, the implementation likely handles only one per button per update tick and ignores the rest.

If your program misses input and you know these two likely causes, you'll quickly track down the source because you only have to look in two places. From there, you can further investigate the reason and possible solutions.

General Software Development

Even if you don't have experience with the particular type of program you're working on, relevant frameworks, or other related technologies, you can still apply your experience from general software development to the problem at hand. For example, if an application freezes, it's highly likely to be related to a deadlock or an infinite loop. If you go through the code to look at all the mutexes, you may very well find the culprit (possibly with the help of a debugger). Alternatively, check all your loops' conditions and make sure they always end. If a program can't write to a file, you know the hard disk may be full or the particular file may be read-only, so you can check that first.

Even if you lack software development experience altogether, you may have some other expertise you can apply. Try treating the program as a black box and observing which inputs cause which issues. These can be anything that influences the program's execution, such as pressing a button, changing a configuration file, or swapping out hardware. Any information about ways to affect the bug lets you make better guesses about its cause.

Focus on Suspicious Systems

Another way to guess where a bug might come from is to investigate suspicious systems. Look for those that have contained multiple bugs before or that are hard to implement without making mistakes.

Check Challenging Systems

Three kinds of systems tend to regularly have bugs across different programs: those that are poorly understood, those that are complex, or those that are used in many different environments.

Poorly understood systems break because the requirements for pre- and post-conditions aren't precise. A programmer is likely to improvise when a system is underspecified, producing code that may not align with the program's general direction or may have problems with usability or design.

Additionally, poorly understood systems often sit between others. If multiple systems interact, you must understand how all of them behave, as well as the requirements for calling into them. For example, a system that connects multi-threaded data processing with the GUI must consider how and when the data processing is ready to accept new inputs and when it can deliver results that the GUI can display. It must also account for processing time, sustained interactivity, and the results of errors or user cancellation.

Very complex systems deal with challenging functionality implementation like network communication, which is hard to get right because it involves a lot of complex pre-conditions for calling each networking function, all results arrive asynchronously, and latency issues arise. If the system sits between a platform's networking API and a program that needs a synchronous interface to get information from some server, these different requirements can cause alignment issues.

Foundational systems also tend to exhibit more bugs than decoupled ones, primarily because people use them more often and in different ways. Consider task schedulers that facilitate multi-threading. Most other systems run through them to parallelize computation, handle dependencies between tasks, and prioritize more urgent work. Just one minor mistake can cause serious bugs in higher-level systems.

One clue that a system's implementation was difficult is heavily commented source code or externals with extensive documentation. If the original programmer thought it necessary to write more about a particular system than the rest of the program, it could signal a higher potential for

bugs. Read comments and documentation carefully because they may give you an indication of what aspects were difficult and where you should focus your search.

Check Repeatedly Broken Systems

If multiple bugs have occurred in a system, there will likely be others, which may be very similar because they stem from the same underlying cause or misunderstanding. When facing a new bug, there are better odds that it lies in a system that had many others before.

Fixing previously existing bugs in a system also increases the probability of creating more because you're making changes on top of the original implementation. Unless you're the system's original developer, you won't know every detail about it, its dependencies, and its requirements. This limited understanding may lead you to a Band-Aid solution that will later turn into another bug. Even the original developers may have forgotten a system's intricacies after a long time away. The more a system grows and changes over time, and the more developers touch it, the more likely it will be a breeding ground for bugs. While this isn't true in all cases, a system that shows many problems is likely to become such a hotspot. It would be wise to address this in the long run, but for now, the goal is just to find an entry point for further investigation.

If you've been working on several bugs for a system, you've probably started to build a bit of knowledge about it even if you're not the original developer. Armed with this information, you can more easily attribute bugs to this system and pinpoint them to a particular spot in the code. Additionally, it should be increasingly straightforward for you to fix bugs in this context. A pattern might emerge over time and point you in the direction of the actual underlying problem.

NOTE

There are many reasons why a developer could make a mistake that leads to a bug, including inexperience with the subject matter and deadline pressure. Alternatively, the system may originally have been created for a specific use case and inappropriately repurposed or implemented as a prototype without anyone producing a proper shippable version. Whatever the reason, assigning blame is never helpful; it's better to work constructively and improve the system.

Initially Ignore Irrelevant Systems

In addition to making educated guesses about the likely source of the bug, it's also useful to rule out areas that you don't need to investigate. The bug's behavior should provide hints about what is or isn't relevant. Start with the seemingly relevant systems and only work outward if you don't find an entry point there.

Look for Relevant Responsibilities

Understanding the responsibilities of the systems in your program and framework can help you determine which systems are likely not related to

your bug. These usually deal with distinct aspects of the program and don't communicate with the functions most likely involved.

Efficiency in bug investigations is one reason why programmers should strive for separation of concerns and decoupling between the various systems that make up a program. When a system's responsibilities are clearly defined, it's easy to tell what functionality it offers and whether that may be involved with your bug. Additionally, if two systems have clear boundaries and interact only when required, you can more easily follow dependency chains.

Systems that take care of different aspects of your program are usually independent. For example, your GUI system probably doesn't directly communicate with the audio system or the networking layer, and the audio likely doesn't directly interact with the networking. Animations are likewise independent of any databases your program might interact with. Intermediate systems may couple these different systems to drive functionality, but they shouldn't communicate directly.

IN PRACTICE: FOCUS ON THE MOST LIKELY CULPRIT

While working on a larger application, I encountered a problem: Not all command line parameters were parsed correctly. Because the architecture of this program was relatively well constructed and the task of parsing parameters didn't depend on a lot of other functionality, I was able to ignore many irrelevant systems (input handling, GUI, audio, and so on).

I started by investigating how the `main()` function parsed the parameters it received from the operating system, using a small system composed of two files. Everything related to strings, containers, and locales also became part of my scrutiny. For example, I thought the logging system might have had an unintended side effect on the strings it was supposed to log, so I kept it in the back of my mind as I worked.

Ultimately, I discovered that the system responsible for parsing parameters wasn't handling positional parameters correctly. I was happy I hadn't wasted my time examining the program's many unrelated systems.

Of course, one or more of the systems you've deemed irrelevant may turn out to be relevant after all. Some systems are connected in surprising ways, influencing each other thanks to unfortunate design decisions, required dependencies that aren't immediately apparent, or shortcuts developers took when implementing them. All programs are unique and may have special requirements that lead to uncommon and potentially surprising relationships. For example, the audio system may pull sample data directly out of a database, the networking layer may write statistics straight to the GUI, or a 3D renderer may request texture data from files directly.

Consider Surprising Dependencies

Bugs can also create relationships between systems that don't have any intentional connection. For example, memory corruption caused by one system usually affects others, changing data to random values in random locations. This can suddenly link two systems that shouldn't logically have anything to do with each other.

Memory corruption can come from multiple factors, such as race conditions, bad pointers, or external sources like files or devices. When it occurs, your program likely won't behave the same every time you run it. A specific type of corruption in a specific place may trigger one bug, while the same problem in another memory location may create a different bug or may not cause problems at all. (For identifying and removing sources of randomness, see “**Remove Randomness**” on [page XX](#).)

Another relatively common reason for independent systems to become connected is the use of generic, foundational features. These might include task schedulers, threading primitives, random number generators, or file access functions. Problems with or misuse of these elements may affect multiple other systems, since they may depend on the same low-level functions.

IN PRACTICE: CHOOSE YOUR RANDOM NUMBER GENERATOR CAREFULLY

If two systems need to generate a random number, each system's sequence should always be independent of the other's. However, if you rely on the good old C function `rand()`, you'll notice that it uses some internal state to generate the next random number, linking any two systems that use this function. Instead, you should choose a random number generator that allows you to pass in the state. Some libraries provide `rand_r(unsigned int*)` for this purpose. Another alternative is to use C++'s generators from the *random* header file.

Know When to Change Your Strategy

As noted earlier, don't spend too long trying out shortcuts. If they don't quickly guide you to the right spot, try other strategies that may take longer but won't lead you to a dead end. As a rule of thumb, you shouldn't spend more than an hour on shortcuts, and usually you'll need less time because your list of guesses and relevant systems tends to be short. It may take a bit longer if you're dealing with a large program that features many potentially relevant systems.

While shortcuts can reveal some information about your bug, they may not provide details about its root cause or potential fixes. In that case, take your knowledge and move on to other strategies. You should try other approaches after probing with two or three shortcuts, or when you feel your

guesses become desperate instead of based on your experience. Remember that you can now cross any system you have investigated with shortcuts off your list of possible culprits.

5. Search the Internet

How? You can use your favorite search engine to seek information related to your bug or try your luck on specialized websites that may offer more applicable content. Don't forget to search your organization's intranet for documentation or discussion about your program.

Why? If you're lucky, someone else will have encountered an identical or similar bug before and may have more information than you've found so far. They might even have posted the solution, which you can adapt for your own purposes. Simply finding more about similar issues or other people's approaches may give you hints or new ideas.

IN PRACTICE: FIND SOLUTIONS TO COMPLEX ISSUES QUICKLY

While working on a little tool designed to send network messages to a larger program, I pulled in a tiny library to help send HTTP requests. This library consisted of precisely one header file. However, after including that file and beginning to build my project, I got the following compilation error:

```
error C2011: 'sockaddr': 'struct' type redefinition
```

It looked like a complex problem that would have taken me down a rabbit hole as I followed type definitions in several header files. Before going that route, however, I put the error message into Google and quickly found the most likely cause: You must include *winsock2.h* before *winsock.h* if you're using both. Indeed, just adding an `#include <winsock2.h>` at the top of my code fixed the problem.

Searching for existing solutions or hints is especially helpful for technologies many people use. Programming languages, tools, services, libraries, frameworks, engines, middleware, SDKs, and APIs may have caused others to run into issues like yours. The more users something has, the more likely you'll find relevant results online.

Solutions to the problem you're trying to solve can come in many shapes. You may find information on how to express something in the programming language you're using, a hint about how to get to a specific command or option in a tool, a new binary build of an SDK, or code you can integrate into your own code base or that of a third-party library.

Internet searches often lead to places that let you ask others for help, like forums, question-and-answer sites, or even social media communities. Do your own research first, but if you can't find any useful information, consider asking others (see “Ask for Help” on page XX). This way, you can get the information you need while contributing to building a shared knowledge base on the internet so that others benefit from questions you've asked or solutions you've posted.

Don't Scroll Too Far

The biggest problem with searching the internet is that it contains a huge amount of information. If you're not dealing with obscure topics or using precise search terms, you could end up with hundreds of millions of hits. Don't waste time sifting through all of them manually.

It's rare to find anything relevant past the second page of search results, so after the first 30 to 50 entries, go back to your terms and options. You should also try a different search engine, ideally one more specialized in the topic you're investigating. If you've already done so, you're probably out of luck and need to turn to a different strategy.

Choose a Search Engine

While the landscape of search engines shifts occasionally, Google has been the most widely used general-purpose option for years. Competitors such as Bing and DuckDuckGo, serve the same needs and you can mostly use them interchangeably. Because searching the internet involves minimal effort, it's best to start with the broadest and most generic engine available. Start with the likes of Google, then try a relatively generic software development knowledge base such as Stack Overflow. Depending on your bug's topic, even Wikipedia may be useful if you need to look up some formula, algorithm, or other fact. While generic searches also include these sites, going directly to the source may give you better results. You can then move on to more specialized websites for the general technology you're working with. For example, you can try the Mozilla Developer Network (<https://developer.mozilla.org>) for anything related to web technologies. If you're working with Microsoft technologies, you may want to try Microsoft Docs (<https://learn.microsoft.com/en-us/docs>).

An answer to your problem could exist on social media, but it may be difficult to find in the overwhelming mass of information. General-purpose search engines often don't index social media data fully or at all, so a dedicated search on social sites may reveal information you can't find otherwise. Try websites like Hacker News (<https://news.ycombinator.com>), Mastodon, Reddit, X (formerly known as Twitter), or YouTube. Their community approach may help you discover highly relevant content.

You can also look for even more specialized sites by using a regular search engine for terms like *web development discord* or consulting the official documentation for non-public sources of knowledge. One of your first stops should be the list of frequently asked questions.

Keep in mind that not all websites are publicly available. Normal search engines may not index some of them, and some may require dedicated accounts to access, particularly if they deal with licensed technology or platforms. If you can make it over this threshold, these communities provide valuable content from experienced users and developers.

A single technology will generally have some or all of the following services:

Bug trackers and feedback systems Repositories that list known bugs and their status. In some cases, they include more general feedback and discussions from which you can gather additional information.

Chat servers Spaces for transient impromptu discussion. Some of these, including Discord and Slack, still offer search functionality.

Documentation and knowledge bases Official documentation for the given technology. These services also provide discussion or editing functionality.

Forums Asynchronous spaces where people can read and reply to content threads. This classic form of online discussion can offer a great deal of information.

Public roadmaps An overview of upcoming updates, features, and bug fixes. Roadmaps are more useful if they incorporate discussion and voting.

Support systems Like bug trackers and forums, but with more focus on community-supported solutions. Each resembles a version of Stack Overflow dedicated to a specific technology.

Compared to general searches, specialized services contain fewer seemingly useful answers that later turn out to be irrelevant. The answers themselves are also likely to be of higher quality, allowing you to use them more directly instead of deriving a solution specific to your situation.

Use Advanced Search Engine Features

No matter what service you're using, first try a very straightforward search with the terms you're looking for. For example, you could look for an error message or a crude description such as *C# disable button*. If that gives you irrelevant results or too much information to sort through, start narrowing down your results with the engine's advanced features.

Sorting options alone might get what you're looking for to float to the top of the list. You could sort by relevance (as the algorithm rates it), time of creation, time of the latest comment, and so on. If you're looking for something older, like information about a bug that existed only in previous versions of a library, it's beneficial to tweak these settings. Filters are also tremendously useful when it comes to reducing the number of results. You may be able to limit your search to a certain section or topic.

Filtering and sorting don't always have convenient user interfaces, so you might need to use operators in the string itself. If you have trouble finding what you're looking for, look into which operators your search engine

supports. Here are some useful operators for Google in particular, most of which can also be used in competing engines either verbatim or with little adjustment:

-term Exclude results containing *term*.

"term" Ensure that *term* actually occurs in the results in this form. It also allows forcing the search engine to list only results where multiple terms appear in the correct order, like "*term1 term2*".

term1 * term2 Match any words or phrases between the two terms.

term1 AND term2 Make sure both terms occur in the results.

term1 OR term2 Allow results that contain one of the two terms.

filetype:pdf Only show results from PDF files.

site:example.com Only show results from the given website.

intitle:term Show results with the given term in the title.

inurl:term Show results with the given term in the URL.

before:YYYY-MM-DD Search for results from before the given date.

after:YYYY-MM-DD Search for results from after the given date.

Be careful to avoid including a space character after operators with a colon or hyphen.

The two most essential operators are *-term* and "*term*". Excluding a word or forcing the results to include one can do wonders. For example, when searching for information related to the Rust programming language, it's helpful to exclude references to the game of the same name: `rust -game`.

IN PRACTICE: USE QUOTES TO GET MORE CONCRETE RESULTS

I once needed to find a file called `px.cpp` that I knew had been previously available online. Just searching for `px.cpp` didn't get me the right results, but when I entered "`px.cpp`" into the search bar, the file appeared on the first page. Similarly, when a dialog box says, *The error code is 2810.*, searching for "*The error code is 2810.*" yields more relevant results than if you leave out the quotes.

Remember that your code may concatenate strings from several partial strings before showing them to you, which means searching for the whole string won't yield any results (see "[Search in All Files](#)" on [page XX](#)). Any concrete numbers, times, values, or general data from your specific instance of the program will most likely have been inserted into the text before being printed to the log, written to a file, or displayed on the screen. Search for partial strings only and consider using the `*` operator to let any value be present.

Search for Source Code

Searching for source code is tricky no matter what engine you use. Many special characters in code either don't show up in regular text or structure that text without carrying any real meaning themselves. Because search engines primarily target regular users trying to find text that isn't code, they ignore those characters. They also don't generally index popular public code repositories: While it's easy to find individual users, organizations, and projects on GitHub, you don't get results for all the code it hosts.

If you really need to find source code, you must go directly to the source, no pun intended. GitHub has a great feature (<https://github.com/search>) that allows searching through all code it hosts with powerful options and filters. While not all online services that host code offer search over all repositories, they typically provide some way to search through specific ones. Find a relevant repository using a regular search engine, then use a specialized query to go through the code. You can also clone an open source repository and search using offline tools if the online options don't help (see “[Search in All Files](#)” on [page XX](#)).

Technologies with generic names make the searching process even harder, especially when they're new. Here are some examples at time of writing:

Go Could mean anything even combined with other terms. As a work-around, search for "golang".

Metal May yield irrelevant results despite the combination. Searching "metal api" can help.

Rust Can be confused with oxidized metal or the game of the same name. Use "rust lang" instead.

Swift A generic term that also has many other specialized uses (for example, in banking or music). Alternative terms are "swift lang" or "swift apple".

Don't Forget Your Intranet

While searching the internet, don't forget its smaller cousin: the intranet. Try looking through any internal wikis, bug trackers, project management tools, and so on. Since they aren't accessible from outside your organization, these tools aren't integrated into any generic search engines. You may even have to query a couple of separate sites, because it's not common for an organization to have an internal search engine covering all the services making up its intranet.

The results will be much more limited, but any information will be highly relevant to your use case and more specialized than anything you can find elsewhere. If a similar bug has occurred before, the documentation left behind from the previous investigation and fix should be especially beneficial (see “[Track Your Debugging Process](#)” on [page XX](#)). Another advantage of finding a source within your organization is that the person who has written down the information may still be in the

office, allowing you to ask follow-up questions if necessary (see “Ask for Help” on page XX).